

LAB SHEET 7

Experiment Name: Customer Segmentation using K-Means Clustering
Steps:

1. Select a suitable dataset such as the Mall Customer Segmentation Dataset, Online Retail Dataset, or Wholesale Customers Dataset.
2. Import the required Python libraries and load the dataset into a Pandas DataFrame.
3. Perform data preprocessing by handling missing values, removing duplicates, and selecting relevant features such as Age, Annual Income, and Spending Score.
4. Normalize or standardize the selected features to ensure that all variables contribute equally during clustering.
5. Determine the optimal number of clusters using the Elbow Method and validate the result using the Silhouette Score.
6. Apply the K-Means Clustering algorithm with the selected value of K to group customers into distinct clusters.
7. Visualize the clusters using scatter plots and highlight the cluster centroids for better interpretation.
8. Analyze the characteristics of each customer segment based on income, spending behavior, or purchasing patterns.
9. Compare the identified customer groups and suggest suitable marketing strategies for each segment.
10. Summarize the clustering results and discuss how customer segmentation can improve business decision-making

Solutions ->

```
STEP 1 and 2 : Import Libraries and Load Dataset

[4]
✓ Os # Step 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
# Step 2: Load CSV directly from URL
url = "https://raw.githubusercontent.com/sharmaroshan/Clustering-of-Mall-Customers/master/Mall_Customers.csv"
df = pd.read_csv(url)
print("Dataset loaded successfully!")
print(df.head())
print("\nDataset Shape:", df.shape)

Dataset loaded successfully!
CustomerID  Genre  Age  Annual Income (k$)  Spending Score (1-100)
0          1    Male   19                15                39
1          2    Male   21                15                81
2          3  Female   20                16                 6
3          4  Female   23                16                77
4          5  Female   31                17                40

Dataset Shape: (200, 5)
```

STEP 3: Data Preprocessing

```
[5]
✓ Os print("\nMissing Values:")
print(df.isnull().sum())
print("\nDuplicate Rows:", df.duplicated().sum())
df = df.drop_duplicates()
features = df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
features = features.dropna()
print("\nSelected Features:")
print(features.head())
```

```
Missing Values:
CustomerID      0
Genre           0
Age             0
Annual Income (k$) 0
Spending Score (1-100) 0
dtype: int64

Duplicate Rows: 0

Selected Features:
Age  Annual Income (k$)  Spending Score (1-100)
0    19                15                 39
1    21                15                 81
2    20                16                 6
3    23                16                 77
4    31                17                 40
```

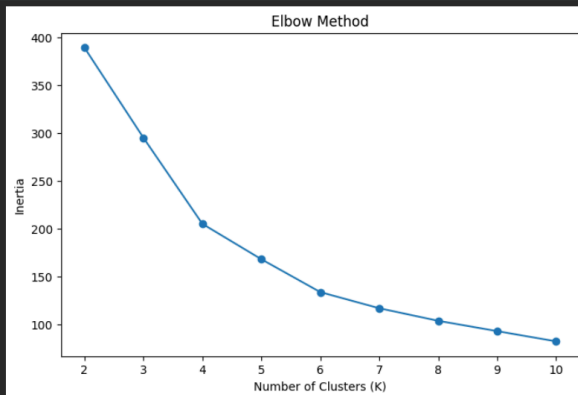
STEP 4: Standardize the Features

```
[6]
✓ Os scaler = StandardScaler()
X = scaler.fit_transform(features)
print("\nStandardized Data:")
print(X[:5])
```

```
Standardized Data:
[[-1.42456879 -1.73899919 -0.43480148]
 [-1.28103541 -1.73899919  1.19570407]
 [-1.3528021  -1.70082976 -1.71591298]
 [-1.13750203 -1.70082976  1.04041783]
 [-0.56336851 -1.66266833 -0.39597992]]
```

STEP 5: Elbow Method

```
[7]
✓ Os inertia = []
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X)
    inertia.append(kmeans.inertia_)
plt.figure(figsize=(8, 5))
plt.plot(range(2, 11), inertia, markers='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.show()
```



STEP 6: Silhouette Score

```
[8]
✓ Os print("Silhouette Scores:")
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(X)
    score = silhouette_score(X, labels)
    print("K =", k, "Score =", round(score, 3))
```

```
Silhouette Scores:
K = 2 Score = 0.335
K = 3 Score = 0.358
K = 4 Score = 0.404
K = 5 Score = 0.417
K = 6 Score = 0.428
K = 7 Score = 0.417
K = 8 Score = 0.408
K = 9 Score = 0.418
K = 10 Score = 0.407
```

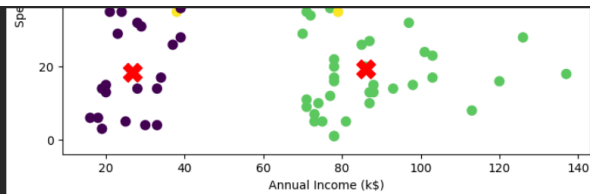
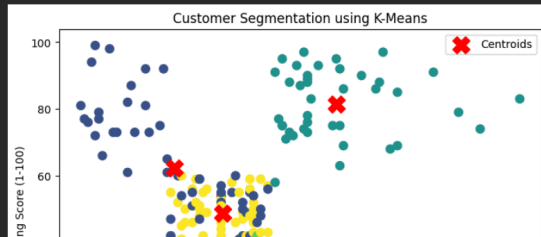
STEP 7: Apply K-Means

```
[9] kmeans = KMeans(
  Os n_clusters=5,
    random_state=42,
    n_init=10
)
clusters = kmeans.fit_predict(X)
features = features.copy()
features['Cluster'] = clusters
print("\nClustered Data:")
print(features.head(10))
```

```
Clustered Data:
  Age  Annual Income (k$)  Spending Score (1-100)  Cluster
0   19                  15                      39         1
1   21                  15                      81         1
2   20                  16                       6         0
3   23                  16                      77         1
4   31                  17                      40         1
5   22                  17                      76         1
6   35                  18                       6         0
7   23                  18                      94         1
8   64                  19                       3         0
9   30                  19                      72         1
```

STEP 8: Visualize Customer Clusters

```
[10] plt.figure(figsize=(8, 6))
  Os plt.scatter(
    features['Annual Income (k$)'],
    features['Spending Score (1-100)'],
    c=features['Cluster'],
    cmap='viridis',
    s=60
)
centers = scaler.inverse_transform(kmeans.cluster_centers_)
plt.scatter(
    centers[:, 1],
    centers[:, 2],
    marker='X',
    s=250,
    c='red',
    label='Centroids'
)
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Customer Segmentation using K-Means")
plt.legend()
plt.show()
```



STEP 9: Analyze Each Customer Segment

```
[11] cluster_analysis = features.groupby('Cluster')[
  Os ['Age', 'Annual Income (k$)', 'Spending Score (1-100)']
    ].mean()
print("\nCustomer Segment Analysis:")
print(cluster_analysis)
```

```
Customer Segment Analysis:
  Age  Annual Income (k$)  Spending Score (1-100)
Cluster
0    46.250000          26.750000          18.350000
1    25.185185          41.092593          62.240741
2    32.875000          86.100000          81.525000
3    39.871795          86.102564          19.358974
4    55.638298          54.382979          48.851864
```

STEP 10: Summary and Conclusion SOLUTION : K-Means clustering successfully divides customers into different groups based on Age, Annual Income, and Spending Score. The Elbow Method and Silhouette Score help determine an appropriate number of clusters. Each cluster represents customers with similar characteristics and spending behavior.

Customer segmentation can help businesses:

Identify high-value customers. Understand different purchasing patterns. Create personalized marketing campaigns. Improve customer retention. Offer targeted discounts and promotions. Allocate marketing resources more effectively.